

TP2 -- Le processus de compilation et les bases du C

Matiere : Decouverte des langages compiles

Niveau : Bachelor 1

Duree : 4h (cours + atelier)

Jour : 2/5 — lundi 18 mai 2026

Objectifs pedagogiques couverts :

- Decomposer la compilation en 4 etapes et observer chaque fichier intermediaire
- Maitriser les options GCC les plus utiles (-Wall, -Wextra, -g, -O2, -c, -S, -E)
- Declarer des variables avec les types primitifs et les utiliser
- Lire des donnees au clavier avec scanf
- Distinguer une erreur de compilation d'un warning

Contexte

Hier, vous avez compile votre premier programme avec `gcc hello.c -o hello`. Mais que se passe-t-il reellement quand vous tapez cette commande ? En realite, GCC execute **4 etapes distinctes** en coulisses. Aujourd'hui, vous allez ouvrir cette boite noire, puis commencer a programmer pour de vrai en C : variables, types, operateurs et entrees clavier.

Partie 1 -- Les 4 etapes de la compilation (1h)

1.1 Vue d'ensemble

```
hello.c
|
v [1. Preprocesseur - gcc -E]
hello.i (macros remplacees, #include inclus)
|
v [2. Compilateur - gcc -S]
hello.s (code assembleur)
|
v [3. Assembleur - gcc -c]
hello.o (code machine binaire, fichier objet)
|
v [4. Editeur de liens - gcc]
hello (executable final)
```

1.2 Exercice : observer chaque etape

Creez le fichier `hello.c` suivant :

```
#include <stdio.h>

#define MESSAGE "Bonjour YNOV !"

int main(void) {
    printf("%s\n", MESSAGE);
    return 0;
}
```

Exécutez chaque commande une par une et observez les fichiers generes :

```
# Etape 1 : Preprocesseur uniquement
gcc -E hello.c -o hello.i

# Etape 2 : Compilation vers assembleur
gcc -S hello.c -o hello.s

# Etape 3 : Assemblage vers fichier objet
gcc -c hello.c -o hello.o

# Etape 4 : Edition de liens (produit l'executable)
gcc hello.o -o hello

# Execution
./hello
```

Questions :

1. Ouvrez `hello.i` : que remarquez-vous par rapport au fichier source original ? Ou est passé `#define MESSAGE` ? Combien de lignes fait le fichier ?
2. Ouvrez `hello.s` : reconnaissez-vous des instructions ? Qu'est-ce que l'assembleur ?
3. Exécutez `file hello.o` puis `file hello` : quelle est la différence entre ces deux fichiers ?
4. Quelle commande unique permet de réaliser les 4 étapes en une seule fois ?

1.3 Focus sur le preprocesseur

Le preprocesseur est un outil de **substitution textuelle**. Il traite les lignes commençant par `#` :

Directive	Role	Exemple
<code>#include <stdio.h></code>	Insere le contenu du fichier header	Fonctions standard
<code>#include "mon.h"</code>	Idem, cherche d'abord dans le dossier courant	Headers perso
<code>#define PI 3.14159</code>	Definit une constante (remplacement textuel)	<code>PI</code> → <code>3.14159</code>
<code>#define MAX(a,b) ((a)>(b)?(a):(b))</code>	Macro avec parametres	Substitution
<code>#ifdef DEBUG</code> / <code>#endif</code>	Compilation conditionnelle	Mode debug

1.4 Focus sur l'editeur de liens (linker)

Quand votre code appelle `printf`, le compilateur ne connaît pas son adresse en memoire. C'est le **linker** qui relie votre code aux bibliotheques (comme la libc).

Erreur frequente : `undefined reference to 'fonction'` signifie que le linker n'a pas trouve l'implementation. Causes : oubli d'un fichier `.o`, oubli de `-lm` pour les fonctions math.

Partie 2 -- GCC en pratique (30 min)

2.1 Les options essentielles

Option	Role	Exemple
<code>-o nom</code>	Definit le nom du fichier produit	<code>gcc main.c -o app</code>
<code>-c</code>	Compile sans lier (produit un .o)	<code>gcc -c main.c</code>
<code>-Wall</code>	Active tous les warnings courants	<code>gcc -Wall main.c</code>
<code>-Wextra</code>	Warnings supplementaires	<code>gcc -Wall -Wextra main.c</code>
<code>-g</code>	Inclut les infos de debogage (pour gdb)	<code>gcc -g main.c -o app</code>
<code>-O2</code>	Optimisations de production	<code>gcc -O2 main.c -o app</code>
<code>-std=c11</code>	Choisit la version du standard C	<code>gcc -std=c11 main.c</code>

Conseil pedagogique : Pendant tout le module, compilez TOUJOURS avec `-Wall -Wextra`. Les warnings vous evitent des heures de debug.

2.2 Exercice : compiler proprement

Creez `test_options.c` :

```
#include <stdio.h>

int main(void) {
    int x;
    int y = 10;
    printf("y = %d\n", y);
    return 0;
}
```

Compilez avec différentes options et observez :

```
gcc test_options.c -o test           # Sans warnings
gcc -Wall test_options.c -o test      # Avec -Wall
gcc -Wall -Wextra test_options.c -o test  # Avec -Wall -Wextra
```

Questions :

1. Quel warning obtenez-vous ? Que signifie-t-il ?
 2. Comment corriger le code pour éliminer le warning ?
 3. Pourquoi est-il important de traiter les warnings même si le programme compile ?
-

Partie 3 -- Variables, types et operateurs (1h)

3.1 Les types primitifs

```
#include <stdio.h>

int main(void) {
    int    age      = 19;      // entier signe (32 bits)
    float  taille    = 1.78f;  // flottant simple precision
    double pi        = 3.14159; // flottant double precision
    char   initiale  = 'A';    // un seul caractere (8 bits)

    printf("Age=%d, Taille=%.2f, Init=%c\n", age, taille, initiale);
    return 0;
}
```

3.2 Tableau des types et formats printf

Type	Taille	Plage approximative	Format printf
char	1 octet	-128 a 127	%c (caractere) ou %d (numerique)
unsigned char	1 octet	0 a 255	%u
short	2 octets	-32 768 a 32 767	%hd
int	4 octets	-2.1 milliards a +2.1 milliards	%d
unsigned int	4 octets	0 a 4.29 milliards	%u
long	8 octets	tres grand	%ld
float	4 octets	~6 chiffres significatifs	%f
double	8 octets	~15 chiffres significatifs	%lf ou %f

Astuce : Utilisez `sizeof(int)` dans votre code pour obtenir la taille réelle sur votre systeme.

3.3 Exercice : explorateur de types

Creez `types.c` qui affiche la taille en octets de chaque type C en utilisant `sizeof` :


```
#include <stdio.h>

int main(void) {
    printf("=== Tailles des types C ===\n");
    printf("char    : %zu octet(s)\n", sizeof(char));
    printf("short   : %zu octet(s)\n", sizeof(short));
    printf("int      : %zu octet(s)\n", sizeof(int));
    printf("long     : %zu octet(s)\n", sizeof(long));
    printf("float    : %zu octet(s)\n", sizeof(float));
    printf("double   : %zu octet(s)\n", sizeof(double));
    return 0;
}
```

Question : Les tailles correspondent-elles au tableau ci-dessus ? Pourquoi peuvent-elles varier d'une machine à l'autre ?

3.4 Les opérateurs en C

Arithmétiques : `+` `-` `*` `/` `%` (modulo) `++` `--`

Comparaison : `==` `!=` `<` `>` `<=` `>=`

Logiques : `&&` (ET) `||` (OU) `!` (NON)

Affectation : `=` `+=` `-=` `*=` `/=`

Attention au piège ! La division entière `7 / 2` vaut `3`, pas `3.5`.
Pour un résultat décimal : `7.0 / 2` ou `(float)7 / 2`.

3.5 Exercice : calculs

Créez `calculs.c` qui :

1. Déclare deux entiers `a = 17` et `b = 5`
2. Affiche leur somme, différence, produit, quotient entier et reste (modulo)
3. Affiche le quotient en flottant (avec un cast)

Sortie attendue :

```
a = 17, b = 5
Somme      : 22
Difference : 12
Produit    : 85
Quotient   : 3 (entier), 3.40 (flottant)
Reste      : 2
```

Partie 4 -- Entrees/sorties avec scanf (30 min)

4.1 Lire des donnees au clavier

```
#include <stdio.h>

int main(void) {
    int    age;
    float  taille;
    char   prenom[30];

    printf("Quel est ton prenom ? ");
    scanf("%29s", prenom);

    printf("Quel age as-tu ? ");
    scanf("%d", &age);

    printf("Quelle taille (en m) ? ");
    scanf("%f", &taille);

    printf("Bonjour %s, %d ans, %.2f m.\n", prenom, age, taille);
    return 0;
}
```

Points importants :

- `scanf` attend l'**adresse memoire** d'une variable : on écrit `&age` (sauf pour les chaines, car un nom de tableau est déjà une adresse)
- Le `%29s` limite l'entree a 29 caracteres pour eviter le debordement (la chaine fait 30 cases)

- Sans le `\n` dans `printf`, le curseur reste sur la meme ligne -- pratique pour les prompts

4.2 Exercice : fiche etudiant interactive

Creez `fiche_interactive.c` qui demande a l'utilisateur :

- Son prenom
- Son age
- Sa note moyenne (float)

Puis affiche un resume formate.

Partie 5 -- Les erreurs en C (30 min)

5.1 Erreurs vs Warnings

Erreurs (errors)	Avertissements (warnings)
Le code ne compile PAS	Le code compile, mais sent mauvais
<code>expected ';' before return</code>	<code>unused variable 'x'</code>
<code>undefined reference to 'printf'</code>	<code>implicit declaration of function</code>
A resoudre AVANT de pouvoir lancer	A traiter serieusement

5.2 Exercice : provoquer et lire des erreurs

Creez un fichier `erreurs.c` et provoquez intentionnellement les 3 erreurs suivantes, une par une. Pour chaque erreur, copiez le message de GCC et expliquez-le :

1. **Oubli du point-virgule** : supprimez un `;`
2. **Type incompatible** : affectez une chaine a un int (`int x = "hello";`)

3. **Fonction non declaree** : appelez une fonction qui n'existe pas
(`maFonction();`)

Partie 6 -- Mini-projet : calculatrice interactive (30 min)

Creez `calculatrice.c` : un programme qui :

1. Affiche "Calculatrice C"
2. Demande a l'utilisateur de saisir un premier nombre (double)
3. Demande un operateur (`+` , `-` , `*` , `/`)
4. Demande un deuxieme nombre (double)
5. Affiche le resultat avec 2 decimales
6. Gere la **division par zero** (affiche une erreur et retourne 1)

Squelette :

```
#include <stdio.h>

int main(void) {
    double a, b;
    char op;

    printf("Calculatrice C\n");
    printf("Operande 1 : ");
    scanf("%lf", &a);
    printf("Operateur (+ - * /) : ");
    scanf(" %c", &op); // espace avant %c : consomme le r
    printf("Operande 2 : ");
    scanf("%lf", &b);

    // Completez : calcul selon l'operateur
    // Gerez la division par zero
    // Affichez le resultat

    return 0;
}
```

Indice : Utilisez une serie de `if` / `else if` pour tester l'operateur.

Pour aller plus loin

- Testez `gcc -v hello.c` pour voir les commandes internes de GCC
- Outil en ligne [Compiler Explorer \(godbolt.org\)](https://godbolt.org) : visualise l'assembleur genere
- Ajoutez le calcul du modulo a votre calculatrice et testez avec des valeurs negatives
- Essayez de compiler avec `-O2` et comparez l'assembleur genere (avec `-S`) par rapport a `-O0`